

Resource-Efficient Speculative Decoding for Long-Context LLM Serving

Fei Li Song Liu Shiqiang Nie Jinyu Wang Weiguo Wu

School of Computer Science and Technology, Xi'an Jiaotong University
Xi'an, China

lifei@stu.xjtu.edu.cn liusong@mail.xjtu.edu.cn shiqiang.nie@xjtu.edu.cn
jinyu.wang@xjtu.edu.cn wgwu@xjtu.edu.cn

Abstract

Speculative decoding reduces sequential Target model calls by verifying multiple tokens from the Draft model in parallel. Yet KV Cache growth limits long-context serving under constrained GPU memory. Offloading KV to CPU memory relieves this pressure. However, existing offloading schemes restore the full KV history before attention and fail to fully exploit the benefits of KV sharing across queries within a verification round. Existing parallel speculative decoding methods also overlook idle GPU compute capacity while memory-bandwidth-bound Target verification waits for historical KV. We present SpecStream, a speculative decoding system that begins verification without waiting for the full KV history to be restored and exploits compute bubbles during KV transfers for concurrent drafting on the same GPUs. It offloads only Target-committed history, keeping candidate rollback local to the GPUs. Each streamed KV chunk serves all queries in the round, with online softmax preserving full attention. Target-priority scheduling controls Draft concurrent execution under resource limits to constrain interference with Target verification. Experiments show that SpecStream maintains task quality close to SGLang speculative decoding while supporting more concurrent requests under limited GPU memory. Across different datasets, it achieves average throughput speedups of $1.41\times$ and $1.32\times$ over the offloading baseline for Qwen3 and InternLM2.5, respectively. Compared with parallel speculative decoding on separate Target and Draft GPUs, SpecStream improves output throughput per GPU by an average of 55.4%.

1 Introduction

Long-context generation imposes a growing memory burden on large language model (LLM) serving. Models typically generate output autoregressively, one token at a time, and each new position attends to the existing context [1]. To avoid recomputing attention Key-Value (KV) for earlier positions, the system retains them in a KV Cache [2]. Model weights remain largely fixed during inference, whereas the KV Cache grows as sequences advance and consumes GPU memory across active requests. Prefill processes input positions in parallel. Decoding, by contrast, advances only a few tokens per forward pass while repeatedly accessing historical KV states. Efficient long-context serving therefore depends not only on compute speed but also on the ability to retain and

access this history efficiently. GPU memory capacity and data movement are central constraints [3].

Speculative decoding [4, 5] accelerates token generation by reducing sequential calls to the Target model, which determines the final output. A lightweight Draft model first generates several candidates. The Target then verifies them in a single forward pass, potentially advancing multiple output positions at once. Candidates become committed output only after acceptance and correction. Recent work improves candidate quality and verification efficiency through trained drafters [6], multi-head proposals [7], and tree-structured verification [8]. Fewer Target calls, however, do not remove the dependence of each verification round on historical KV states. The weights and KV Caches of both models, candidate states, and execution buffers all compete for GPU memory. Speculative decoding thus shortens the sequential execution path but may also reach the GPU memory limit sooner.

Offloading the KV history to larger CPU memory provides a direct way to relieve GPU memory pressure. Existing systems improve KV management through tiered storage [3], cache reuse [9], and selective retrieval [10]. In speculative decoding, migration must also account for the Target’s commit state. Otherwise, the system may transfer and maintain KV states for candidates that are later rejected. Restricting migration to committed history avoids this wasted work, but verification still has to wait for data. If every round restores the full history before computation begins, transfer time grows with context length and extends the critical path of Target verification. Offloading therefore requires more than a change in storage location. The Target must be able to compute with incoming KV states promptly and reuse them effectively.

KV access in speculative decoding differs from that in single-token autoregressive decoding. The queries in a Target verification round have different causal visibility within the candidate region but share the same earlier committed KV history. A single history fetch can therefore serve the entire verification query group, amortizing its transfer cost over the tokens actually committed in that round. Optimizing historical KV access therefore requires more than faster transfers; it must also account for the useful output each transfer supports. In addition, KV offloading paths in existing speculative decoding systems [25, 36] still defer attention computation until the current layer’s full KV history is available on the GPU, preventing already transferred chunks from being processed as they arrive.

Waiting for historical KV leaves GPU compute capacity idle during memory-bandwidth-bound Target verification. With historical KV offloaded to CPU memory, verification requires host-to-device (H2D) transfers, which can further extend these waits. Even with partial overlap between transfers and computation, the Target can still stall if the current computation finishes before the next KV chunk arrives, creating compute bubbles. MineDraft [11], SPECTRE [12], and Swift-Spec [13] hide part of the drafting latency by overlapping candidate generation with Target verification through cross-GPU parallelism or pipelined scheduling. These designs, however, do not directly exploit these bubbles to advance the Draft on the Target GPUs.

These observations motivate SpecStream, which jointly optimizes historical KV access and speculative execution. SpecStream uses the Target commit boundary to constrain history migration, keeping candidate rollback local to the GPUs. During Target verification, SpecStream combines chunked transfers, KV sharing across queries, and online softmax to let computation advance as history chunks arrive, without waiting for the current layer’s full history to be restored, while preserving full causal attention. Dynamic verification width further balances the benefits of history reuse against additional computation. Building on this design, SpecStream schedules Draft co-execution on the same GPUs under a Target latency budget, using spare compute capacity during transfers to advance candidate generation. This reduces the drafting latency exposed on the critical path and the reliance on additional GPUs. The contributions are summarized as follows:

- We propose commit-aware rollback-free KV tiering. SpecStream uses the Target commit boundary to constrain migration, offloading irrevocably committed older history to CPU memory while retaining recent KV states and unverified candidates on the GPU. History migration and speculative rollback operate on disjoint state regions. Batched migration also reduces fragmented transfers.
- We propose KV streaming for multi-query verification. SpecStream divides the offloaded KV into contiguous chunks. Each chunk serves every query in the current verification round after it reaches the GPU, and online softmax combines the history chunks with recent GPU-resident KV states to compute full causal attention. Grouped transfers, double buffering, and cross-layer prefetching overlap data movement with computation. Cohort batching aggregates compatible tasks within a batch, while the verification width adapts to transfer cost, candidate acceptance, and compute overhead to improve the efficiency of KV history transfers.
- We propose Target-priority intra-GPU co-execution. SpecStream combines offline measurements with runtime observations to estimate available execution windows under the Texture Processing Cluster (TPC) allocation selected for the Draft. It schedules subsequent candidate generation one token at a time. Completion feedback updates execu-

tion estimates and admission decisions for the next step, allowing the Draft to use spare compute capacity during KV history transfers while controlling interference with Target computation.

- We implement SpecStream in SGLang and evaluate it with Qwen3 and InternLM2.5. Across three different datasets, SpecStream supports more concurrent requests under limited GPU memory than methods without offloading and achieves average throughput speedups of $1.41\times$ and $1.32\times$, respectively, over the offloading baseline. Its shared-GPU execution improves output throughput per GPU by an average of 55.4% compared with speculative decoding that places the two models on separate GPUs.

2 Background

2.1 Speculative Decoding

Standard autoregressive decoding generates one new token per round. Speculative decoding introduces a computationally cheaper Draft model so that a single Target verification pass can advance multiple output positions [4, 5]. At the start of round r , let the length of the confirmed output prefix be N_r , and denote it by $x_{1:N_r}$. Let q denote the verification width. Conditioned on this prefix, the Draft generates q consecutive candidate tokens as follows:

$$\tilde{x}_{N_r+i} \sim p_D(\cdot \mid x_{1:N_r}, \tilde{x}_{N_r+1:N_r+i-1}), \quad 1 \leq i \leq q. \quad (1)$$

Here, p_D is the Draft’s conditional next-token distribution.

A single Target forward pass produces the Target distributions for the candidate positions. The acceptance and correction rules then determine the committed output. Let $U_r(q)$ denote the number of output tokens committed in this round. Ignoring changes in batching and assuming sequential execution of drafting and verification, the mean execution cost per committed token is approximated by

$$\bar{T}_{SD}(q) = \frac{T_D(q) + T_V(q) + T_R(q)}{\mathbb{E}[U(q)]}. \quad (2)$$

Here, T_D , T_V , and T_R are the mean times spent on drafting, Target verification, and correction after rejection under the measured workload. Progress is counted in terms of the output tokens ultimately committed, rather than the number of accepted Draft candidates. Increasing q may advance more tokens per round, but it will also increase the work spent on candidate generation and verification, including computation for rejected candidates.

2.2 KV Caches in Speculative Decoding

Tokens produced by standard autoregressive decoding become valid history for subsequent positions. In speculative decoding, the Target KV Cache contains both committed and unverified states. During round r , let C_r be the length of the materialized KV prefix corresponding to committed output,

and let L_r be the logical KV length including current candidates. Newly generated output tokens whose KV states have not yet been computed do not count toward the committed KV boundary. The state can then be written as $\mathcal{K}_r^{\text{comm}} = [0, C_r)$ and $\mathcal{K}_r^{\text{spec}} = [C_r, L_r)$. Here, $\mathcal{K}_r^{\text{comm}}$ denotes committed history and $\mathcal{K}_r^{\text{spec}}$ denotes unverified candidates. If the committed KV prefix retained at the end of the round has length C_{r+1} , the Target KV interval to reclaim is $\mathcal{R}_r = [C_{r+1}, L_r)$. Committed KV history therefore remains valid, whereas part of the speculative frontier may be discarded. KV offloading must distinguish these two lifetimes. Consider a model with L layers, n_{kv} KV heads per layer, head dimension d_h , and b_e bytes per element. For B concurrently executed requests of length C , the required Target KV capacity is approximately $M_{\text{KV}}(B, C) \approx 2BLn_{\text{kv}}d_hb_eC$. The factor of 2 accounts for keys and values. Speculative decoding must additionally retain the Draft model’s weights and KV Cache, as well as Target KV states for the current candidates. These states further increase memory demand at long contexts and high concurrency.

3 Motivation

3.1 KV Sharing in Multi-Query Verification

With KV history offloaded to CPU memory, the cost of speculative decoding depends on both the amount of KV transferred per round and the useful output that each transfer supports. For a given request, autoregressive decoding processes one new query per step, whereas a speculative Target forward pass verifies q positions at once. These queries have different causal visibility within the candidate region but all access the same committed KV history. A single copy of that history can therefore supply the entire verification query group.

Sharing within a round creates an opportunity to amortize history transfers. For a fixed history length, let D_H be the amount of CPU-resident history read in one round and $U(q)$ the number of tokens committed. The average history transfer volume per committed token is approximately

$$D_{\text{token}}(q) \approx \frac{D_H}{\mathbb{E}[U(q)]}. \quad (3)$$

The history size is largely independent of verification width, but one verification pass may advance several output tokens. Speculative verification thus reduces sequential Target calls and lets multiple useful outputs share the cost of a history transfer. Verification width affects both computation and transfer amortization. As the accepted length approaches saturation, increasing q provides less transfer benefit while still adding computation. Long-context speculative serving must therefore consider history access together with the full verification round, weighing reuse against additional work in terms of cost per useful output token.

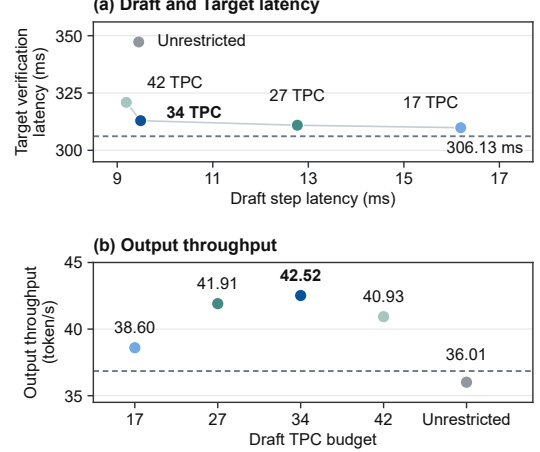


Figure 1: Draft GPU resource allocation and inference performance with KV offloading. TPC [16] masks restrict the Draft to selected GPU compute clusters. (a) Single-token Draft latency versus Target verification latency under different TPC allocations. (b) Corresponding output throughput. The model pair is Qwen3-32B/0.6B [14], evaluated on LongBench v2. Dashed lines show the Target verification latency and end-to-end throughput of the Draft and Target serial execution. Unrestricted denotes concurrent execution on the same GPUs without a TPC limit on the Draft model.

3.2 Parallel Speculation with KV Offloading

Parallel speculation often uses separate GPUs to overlap Draft generation with Target verification [11–13]. However, memory-intensive decoding can leave spare compute capacity on the Target GPUs that these designs may not fully exploit.

KV offloading creates such an opportunity. With history stored in CPU memory, Target verification waits for the required data to arrive on the GPU. Even when transfers and computation overlap in part, the compute stream stalls if attention on the current chunk finishes before the next chunk is ready. For a stage in which transfer and computation can overlap, let T_{H2D} denote history transfer time and T_{attn} the attention time that can overlap it. The resulting compute wait is approximated by $G \approx \max(0, T_{H2D} - T_{\text{attn}})$.

Compared with keeping the entire KV Cache on the GPU, offloading introduces additional bubbles whenever H2D transfers are not hidden by computation. The copy engine is still moving data, but some compute units are idle. The models consequently have complementary resource demands during these stages. The Target is limited by data arrival, while the Draft model still has candidates to generate. Advancing the Draft model while the Target waits for history can hide part of the drafting cost within the current verification pass.

Only part of the compute capacity is available during a bubble. H2D transfers still write to GPU memory, and Target computation must resume promptly once the data arrives. Giving the Draft too many resources may interfere with transfers

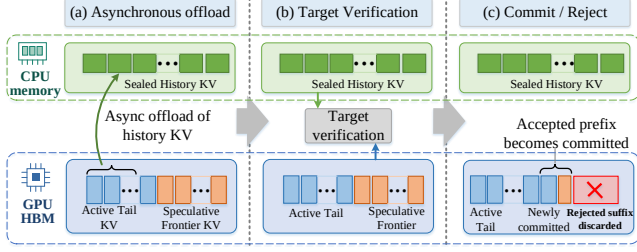


Figure 2: Rollback-free KV state tiering.

or delay subsequent Target kernels. Too few resources, however, may lengthen a Draft step beyond the available window. Shared-GPU execution must therefore match both the duration and resource demand of drafting to the spare capacity left by KV transfers. Only then can temporary idle capacity translate into Draft progress.

Figure 1 illustrates this resource trade-off. In Fig. 1(a), increasing the Draft allocation shortens candidate generation but lengthens Target verification. With no allocation limit, the Draft step takes about as long as in some constrained configurations, yet interference with the Target is much greater. Overall throughput in Fig. 1(b) first rises and then falls; unrestricted drafting even performs below the offloading reference. Faster drafting improves overall throughput only when its benefit outweighs the interference imposed on the Target. Shared-GPU scheduling must coordinate the Draft resource allocation, start time, and work admitted per invocation to fit the available execution window.

4 Methodology

SpecStream jointly designs Target KV storage management, data access during verification, and intra-GPU scheduling. It comprises three components. A commit-aware KV tiering manager identifies the range eligible for migration and coordinates asynchronous copies with GPU space reclamation. A multi-query streaming verifier allows each history block to serve the entire query group and computes full attention through online softmax. It combines a transfer pipeline, Cohorts, and dynamic verification widths to improve execution efficiency. A Target-priority scheduler controls when the Draft model starts and how far it advances in parallel under the selected TPC allocation. During each round, the Target model reads the CPU-resident history and recent GPU-resident KV layer by layer, then updates the commit state after verification. The manager subsequently offloads eligible older history, while the intra-GPU scheduler advances the Draft model in permitted phases. The controller uses runtime feedback to adjust the verification width for the next round.

4.1 Commit-Aware Rollback-Free KV Tiering

Speculative decoding assigns different lifetimes to KV states. SpecStream offloads historical KV without extending candidate rollback across devices. It aligns migration eligibil-

ity with the Target model’s commit boundary: offloaded KV belongs to an irrevocably committed prefix, while KV for unverified candidates remains on the GPU. History migration and candidate rollback therefore operate on disjoint state regions, avoiding transfers of KV for candidates that may later be rejected and the need for CPU-side cleanup after rejection. Fig. 2 illustrates this partitioning.

For each request, let H denote the length of historical KV whose migration to the CPU has completed, C_r the length of the committed KV prefix, and L_r the logical length including the current candidate KV. The Target KV Cache is partitioned into the following three regions accordingly,

$$\begin{aligned} \text{Sealed History:} & \quad [0, H) \text{ (CPU)} \\ \text{Active Tail:} & \quad [H, C_r) \text{ (GPU)} \quad 0 \leq H \leq C_r \leq L_r. \quad (4) \\ \text{Speculative Frontier:} & \quad [C_r, L_r) \text{ (GPU)} \end{aligned}$$

Sealed History stores committed historical KV whose migration has been completed. Active Tail retains recently committed KV, while Speculative Frontier contains candidate KV awaiting verification. The commit boundary C_r constrains the migration range, ensuring that the CPU holds only history that remains valid. Candidate acceptance, rejection, and the resulting KV reclamation are all handled on the GPU.

Committed KV need not be offloaded immediately. Offloading the KV of every newly committed token would produce many small, fragmented transfers and increase synchronization and metadata-maintenance costs. SpecStream instead retains an Active Tail with a target length of T on the GPU. Newly committed KV first accumulates in this recent history, after which older contiguous KV blocks are migrated to the CPU in batches. Let the migration granularity be g tokens, with H already aligned to this granularity. The endpoint of the current migration is

$$E = \max \left\{ H, g \left\lceil \frac{\max(0, C_r - T)}{g} \right\rceil \right\}. \quad (5)$$

If $E > H$, the system migrates the interval $[H, E)$; otherwise, it leaves the state unchanged. Migration uses asynchronous copies. The history boundary advances to E only after the copy has completed and the data is confirmed to be available; the corresponding GPU space can then be reclaimed. The commit boundary C_r determines which KV is eligible for migration, the recent-history retention target T controls the GPU tail size, and the migration granularity g controls the transfer batch size.

The verification result then determines how the speculative frontier is updated. Let C'_r be the length of the committed KV prefix retained at the end of the round. Accepted candidates move into Active Tail, and the suffix $[C'_r, L_r)$ is released through the existing GPU KV reclamation procedure. Because later candidate rejections cannot invalidate an already committed prefix, $H \leq C_r \leq C'_r$ always holds. Hence, $[C'_r, L_r) \cap [0, H) = \emptyset$.

The rollback range is disjoint from the CPU historical KV. Candidate rejection changes only the speculative suffix on the GPU, without requiring CPU replica cleanup or cross-device rollback. As the commit boundary advances, accepted KV gradually becomes older history and is migrated according to Eq. (5). Using the same KV state boundaries, we next describe how CPU-resident history participates in Target verification.

4.2 KV Streaming for Multi-Query Verification

SpecStream allows Target verification to begin before the current layer’s full KV history is restored. It uses the verification query group as the unit of history reuse and the KV chunk as the unit of transfer and computation. Combined with online softmax, this organization preserves full causal attention while allowing arriving history to be processed before subsequent chunk transfers complete. Double buffering and cross-layer prefetching hide part of the H2D cost, while Cohort batching reduces scheduling overhead. The system also dynamically adjusts the verification width based on the estimated time per committed token, balancing transfer amortization against additional computation. Fig. 3 shows the design.

Chunked verification shared across queries. Although the q queries in a verification round occupy different positions, they can all access the earlier Sealed History. Historical KV can therefore serve as a shared input to the entire query group, without repeated transfers for different verification positions. Visibility within the candidate region remains governed by each query’s causal dependencies.

SpecStream partitions the Sealed History of the current layer into J contiguous chunks along token positions. Let $Q \in \mathbb{R}^{q \times d_h}$ be the query matrix for the round, where d_h is the head dimension. Once chunk j , denoted by (K_j, V_j) , arrives on the GPU, the verifier computes its attention scores using the entire query group as $S_j = QK_j^T / \sqrt{d_h}$.

A chunk needs to be transferred only once for the current layer and round to serve all q verification queries. After the entire query group has finished processing it, the staging buffer can receive a subsequent chunk. Verification progresses as historical chunks arrive, without waiting for the full history of the current layer to be restored to the GPU.

The per-chunk results must also be combined into full attention. Denote the complete CPU-resident history by K_H, V_H and the GPU-resident Active Tail and Speculative Frontier by K_G, V_G . The computation seeks to obtain

$$O = \text{softmax} \left(\frac{Q[K_H; K_G]^T}{\sqrt{d_h}} + M \right) [V_H; V_G]. \quad (6)$$

Here, M is the causal mask. For each query, all visible KV positions jointly determine the softmax normalization denominator. Normalizing each chunk independently and then summing its output would assign each chunk a different denominator, changing its relative weight in the full history. The

verifier must therefore retain intermediate states that have not undergone final normalization and can be merged across chunks.

SpecStream uses online softmax to maintain normalization state chunk by chunk [17, 18]. For each query and attention head, it stores three accumulators (m, ℓ, a) . Here, m is the maximum attention score over the KV positions processed so far, ℓ is the sum of exponentials relative to that maximum, and a is the weighted sum of values using the same exponential weights. The latter two accumulate the denominator and numerator of the final output, respectively, without performing the division during intermediate computation. The initial state is $(-\infty, 0, 0)$.

For each newly arriving chunk j , let S_j denote its attention scores and V_j its values. The following updates are performed separately for each query and attention head, with k ranging over the KV positions in the current chunk that are visible to that query. The verifier first computes the local state within the chunk:

$$m_j = \max(S_j), \ell_j = \sum_k e^{S_{j,k} - m_j}, a_j = \sum_k e^{S_{j,k} - m_j} V_{j,k}. \quad (7)$$

Here, m_j is the maximum score in the current chunk, while ℓ_j and a_j are its local sum of exponentials and weighted sum of values, both relative to m_j . The accumulated state and the current chunk may use different maxima, so they must be rescaled to a common reference before merging. Let

$$m' = \max(m, m_j), \alpha = e^{m - m'}, \beta = e^{m_j - m'}. \quad (8)$$

The accumulated denominator and weighted sum of values are then updated as $\ell' = \alpha\ell + \beta\ell_j$ and $a' = \alpha a + \beta a_j$.

The scaling factors α and β align the existing state and the current chunk’s local state to the common maximum m' . Because the numerator and denominator use the same scaling factors, merging preserves the relative weights of previously processed KV in the full softmax. After processing each chunk, the system sets $(m, \ell, a) \leftarrow (m', \ell', a')$ and proceeds to the next chunk.

After processing all CPU history chunks, the verifier continues with Active Tail and Speculative Frontier on the GPU. The earlier CPU-resident history is visible to the entire query group. The GPU-resident region, however, requires a causal mask based on each query’s absolute position to exclude subsequent candidates that it cannot see. These results update the accumulated state in the same manner. Once all causally visible KV has been processed, the attention output is $O = a/\ell$.

Each chunk’s result has been rescaled to a common maximum during merging. The final accumulators therefore satisfy

$$\frac{a}{\ell} = \frac{\sum_{t \in \mathcal{V}} e^{s_t - m}}{\sum_{t \in \mathcal{V}} e^{s_t - m}}. \quad (9)$$

Here, $\mathcal{V}$ denotes all KV positions causally visible to the current query. This ratio uses a numerator and denominator jointly formed over all visible positions, making the chunked reduction equivalent to computing the full softmax at

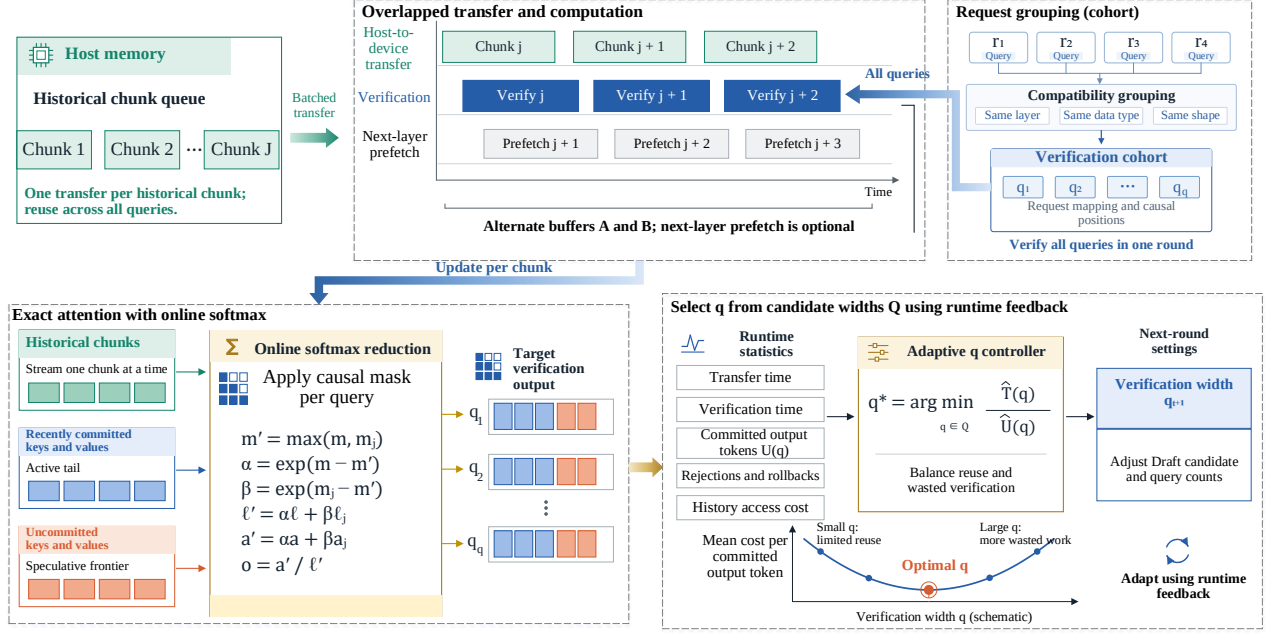


Figure 3: Exact streaming execution for multi-query verification.

once in real arithmetic [17]. SpecStream changes only the order in which KV arrives on the GPU and participates in the reduction. It preserves the definition of full-history attention.

Pipelined KV transfers and prefetching. Chunked reads shorten the wait before computation can begin. Yet if every chunk is processed serially, with each transfer followed by computation and the next transfer starting only afterward, the waiting time still accumulates across chunks. SpecStream uses two GPU staging buffers and separate transfer and compute streams to overlap H2D transfers with attention computation. While the GPU processes chunk j in buffer A, buffer B receives chunk $j+1$. Once computation on the current chunk finishes, A receives subsequent data. Adjacent chunks are grouped into transfer batches to amortize copy-submission, synchronization, and kernel-launch overheads. Once the current layer releases a reusable buffer, the system prefetches historical KV for the next layer, overlapping the transfer with the current layer’s remaining attention, output projection, and feed-forward computation to reduce waits in the next layer.

Cohort batching under high request concurrency. When multiple requests enter verification together, processing historical chunks separately for each request creates many fine-grained tasks. Each request requires data preparation, transfer submission, and attention kernel launches at every layer. Even if an individual task overlaps transfer with computation, preparation and synchronization costs still accumulate with the request count, and small compute tasks may underutilize the GPU. SpecStream constructs Cohorts within the current scheduled batch, grouping tasks that can be processed by the same batched kernel. Compatibility requires matching model layers, verification widths, data types, numbers of KV

heads, head dimensions, and chunk sizes. The system prepares historical chunks and queries by group, then executes each request’s computation through a batched attention kernel. Requests retain independent KV indices, absolute positions, and online softmax states, and each result updates its own accumulators. This increases the parallelism of each kernel invocation while amortizing data preparation, synchronization, and kernel-launch costs.

I/O-aware dynamic verification widths. Sharing historical KV across queries makes the verification width q determine both the candidate count and the granularity of history reuse. The benefit of reuse also depends on the number of tokens actually committed. Following the transfer amortization relationship in Eq. (3), SpecStream dynamically selects q using candidate acceptance and execution costs.

With a long KV history and a small verification width, each read is amortized over only a few output tokens. A moderate increase in q allows the same history to serve more potentially useful outputs; when transfers are the bottleneck, the additional attention computation may also be hidden by transfer time. Once the accepted length approaches saturation, this benefit diminishes even as candidate generation and verification work continue to grow. The controller therefore selects q by the estimated time per useful output token. It uses recent history-transfer, verification, Draft-generation, and rejection-correction costs, together with the number of tokens actually committed, to estimate the round cost at different widths and selects $q^* = \arg \min_{q \in Q} (\hat{T}_{\text{round}}(q) / \hat{U}(q))$. Here, Q is the set of candidate verification widths, and $\hat{T}_{\text{round}}(q)$ and $\hat{U}(q)$ denote the estimated round time and number of committed tokens, respectively.

The controller records Draft execution time and useful progress for each width, prioritizing existing observations. When records are insufficient, it estimates costs from available measurements and collects information for each candidate width through a limited initial sampling phase. Verification costs are estimated from the historical data volume, transfer rate, and chunk computation time, without counting overlapped time twice. The system updates its statistics at the end of each round and changes the width for the next round only when the estimated benefit exceeds the switching threshold. Thus, q adapts to history-access costs, batching conditions, and candidate quality while limiting frequent switching.

4.3 Target-Priority Intra-GPU Co-Execution

SpecStream turns the compute bubbles caused by Target KV-transfer waits into candidate-generation progress on the same GPU. This enables parallel speculation without provisioning additional compute resources for the Draft model. The scheduler uses the Target model’s available time and latency budget to govern Draft execution, rather than simply pursuing the fastest possible drafting. An offline-selected TPC allocation limits Draft resource use, while runtime admission determines when the allocated resources can be used within the remaining window. Fig. 4 shows the design.

Draft compute allocation. SpecStream uses NVIDIA Multi-Process Service (MPS) [19] to enable concurrent execution of the Target and Draft models in separate processes. MPS allows kernel execution and data copying from different CUDA processes to overlap on the same GPU, providing an environment for the two models to execute concurrently. For resource control, SpecStream limits the Draft model’s compute resources at TPC granularity. A TPC is a hardware cluster containing Streaming Multiprocessors (SMs) [20]. The system applies a TPC mask in the separate Draft process so that its compute kernels run only on the designated set of TPCs [16], while preserving the Target model’s original access to compute resources. TPC restrictions do not isolate the L2 cache or memory bandwidth, so resource contention between the models must still be controlled at runtime.

At deployment, SpecStream first measures the execution performance of both models under different TPC allocations. With fewer TPCs, the Draft model takes longer to generate one token; with more, it may prolong Target verification. The system defines a runtime configuration x by the model pair, batch size, context length, verification width, and execution phase. It records the Draft model’s single-step time $t_D(x, n)$ with n TPCs and computes the increase in Target verification latency relative to execution alone as $\Delta_T(x, n) = [T_T^{\text{co}}(x, n) - T_T^{\text{alone}}(x)] / T_T^{\text{alone}}(x)$. Here, $T_T^{\text{co}}(x, n)$ and $T_T^{\text{alone}}(x)$ are the Target verification latencies during co-execution with the Draft model and during standalone execution, respectively. Both include the wait for historical KV. The Draft model’s single-step time is also measured during co-execution to capture the effect of contention on candidate

generation. The offline sweep additionally compares end-to-end throughput across allocations and selects a default TPC allocation n_0 . At runtime, the system determines whether the Draft model may start under this allocation.

Target-priority scheduling. At runtime, SpecStream uses the transfer wait, verification-phase durations, and Draft single-step time observed in the preceding round to update the corresponding time estimates for the next round. It then estimates the remaining time available to the Draft model, $\hat{S}$, from current progress. Starting from the offline records, the system incorporates runtime feedback to update the estimated Draft single-step time $\hat{t}_D(x, n_0)$ and Target latency increase $\hat{\Delta}_T(x, n_0)$ under the selected allocation. Let ϵ_T be the acceptable increase in Target latency and δ a time margin reserved for execution variability. Starting a Draft step under allocation n_0 must satisfy conditions: $\hat{\Delta}_T(x, n_0) \leq \epsilon_T$ and $\hat{t}_D(x, n_0) + \delta \leq \hat{S}$. The first condition bounds the estimated increase in Target latency. The second requires the estimated duration of one Draft step, including the reserved margin, to fit within the remaining window. The scheduler submits a new concurrent step only when both conditions hold; otherwise, it defers execution.

The available window changes as the Target model advances. The scheduler therefore admits only one full Draft forward pass for a single token at a time. Longer windows can accommodate several successive steps; shorter ones permit fewer steps or none. Single-token admission bounds the in-flight work that cannot be revoked immediately, reducing the risk that Draft execution outlasts the waiting window and delays Target computation.

Each scheduling instruction carries the request and speculative-round identifiers, an execution sequence number, the TPC allocation, and the latest start time. If the instruction is issued at t_0 with an estimated remaining time of $\hat{S}$, its latest start time is $t_{\text{latest}} = t_0 + \hat{S} - \hat{t}_D(x, n_0) - \delta$. Before starting, the Draft model checks that the instruction still matches the current request and speculative round and that the latest start time has not passed. Expired or mismatched instructions are not executed. After completing the GPU computation for one token, the Draft model returns an acknowledgment (ACK), with the execution sequence number linking the completion result to the corresponding instruction. Only after receiving the ACK does the Target model decide whether to schedule another step based on the remaining time, keeping at most one unfinished Draft step per request.

Execution results are then used to update the estimated Draft single-step time and Target latency. When the dynamic verification-width controller in Section 4.2 changes q , or when the batch size or context length changes, the system reads the measurement records for the default allocation under the new runtime configuration. If the observed increase in Target latency exceeds the budget, the scheduler stops submitting new concurrent steps and restores exclusive Target execution once the already launched Draft computation finishes. If the Tar-

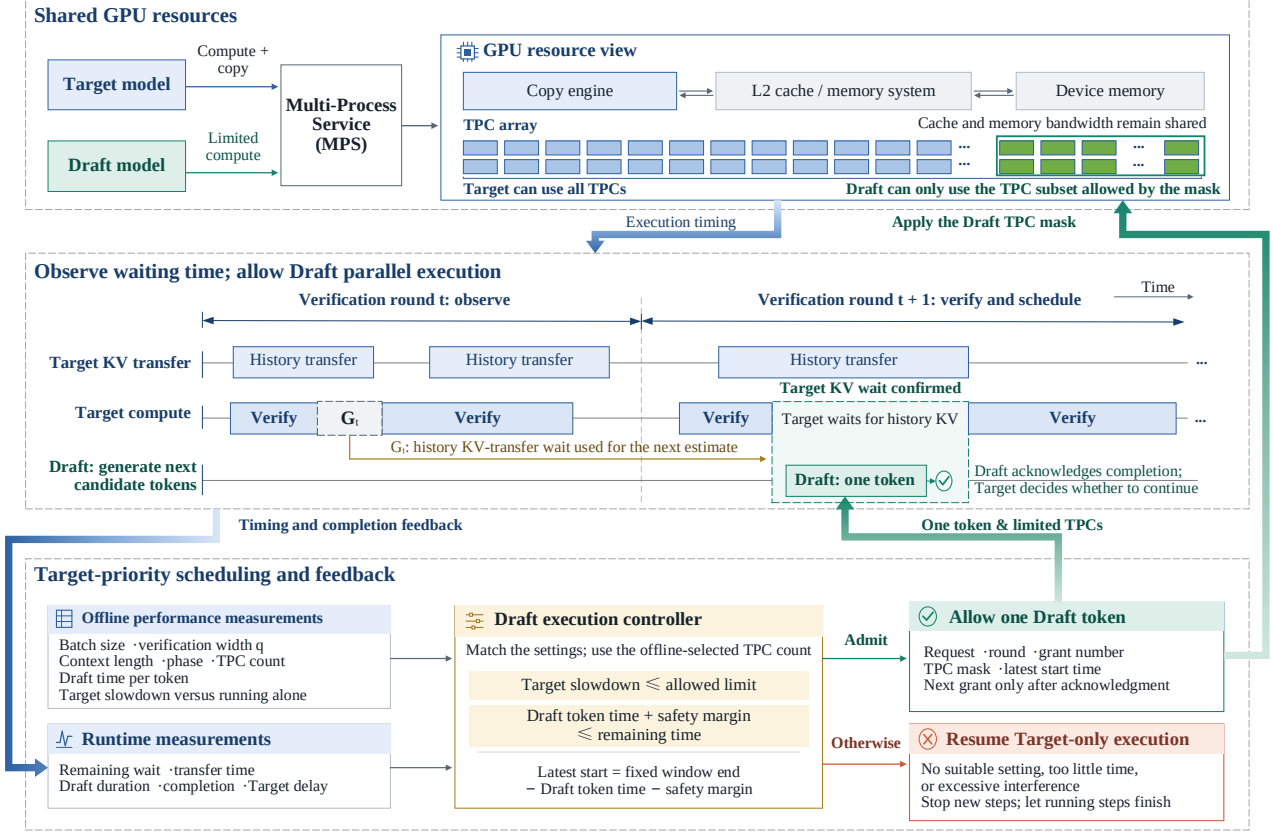


Figure 4: Target-Priority Parallel Speculation on Shared GPUs.

get model is already waiting for candidate tokens required by the current round, the request continues through the original Draft-generation procedure. Whether parallel execution resumes is determined by the Target model’s current state.

5 Experimental Results

5.1 Experimental Setup

Models and deployment. We implement SpecStream on SGLang 0.5.17 [21] and evaluate two Target/Draft model pairs, Qwen3-32B/0.6B [14] and InternLM2.5-20B-Chat/1.8B-Chat [15]. The main experiments use two NVIDIA A800 80GB PCIe Gen4 GPUs and 240GB of host memory. SpecStream’s Target and Draft models share these GPUs, both using tensor parallelism (TP) with degree 2, and use separate process groups and MPS to support concurrent execution. The separate-GPU parallel baseline in Section 5.3.3 uses three A800 GPUs. Two run the Target with TP=2, and the third runs a dedicated Draft model with TP=1.

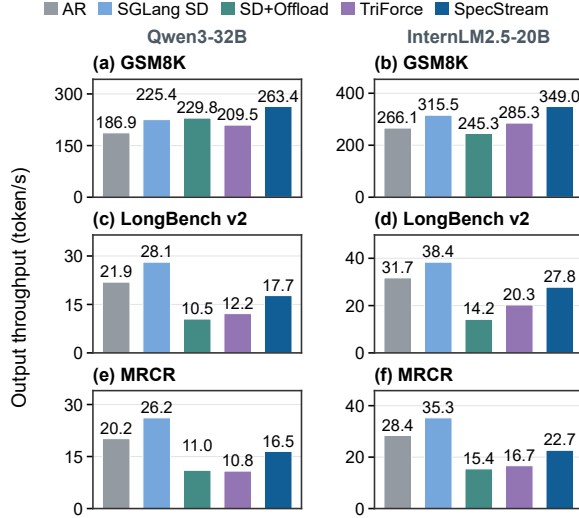
Workloads. GSM8K [22] evaluates reasoning with short inputs, while LongBench v2 [23] and MRCL [24] cover long-document understanding and retrieval from long conversations, respectively. The component ablations use token-ID inputs generated with a fixed random seed to control input length and the number of concurrent requests.

Baselines. We compare SpecStream with autoregressive decoding (AR), SGLang speculative decoding (SGLang SD) [21], SGLang SD+Offload (SD+Offload), and TriForce [25]. SD+Offload uses chunked KV transfers, double buffering, and cross-layer prefetching, but starts attention only after the layer’s full KV history is restored to the GPU. TriForce combines a Draft model with a bounded KV window, retrieval-based intermediate speculation, and verification against the full history. Because its public implementation targets Llama models, we adapt the original method to Qwen3 and InternLM2.5. Related systems emphasize different designs: VeriCache [36] uses compressed-KV self-speculation, SpecOffload [37] targets weight-offloaded inference, and Dovetail [38] performs Target verification on the CPU. We select TriForce as a representative offloading baseline for our setting. SGLang SD and SD+Offload use a fixed verification width of $q = 4$, whereas SpecStream dynamically selects q from $\{2, 4, 6, 8\}$. Unless otherwise specified, SpecStream retains a 512-token Active Tail and allocates no additional GPU cache for historical Target KV. For parallel speculation, we also compare against SPECTRE+Offload, which uses SPECTRE’s [12] parallel execution mechanism across separate GPUs, together with our history offloading and streaming verification path.

Metrics. Our primary performance metric is end-to-end output throughput in tokens/s. Task quality is measured using

Table 1: Task-quality scores of SpecStream and SGLang SD.

Model	Dataset	SGLang SD (%)	SpecStream (%)	Difference (pp)
Qwen3	GSM8K	95.2237	95.1479	-0.0758
	LongBench v2	45.8015	45.8015	0.0000
	MRCR	55.4946	55.6571	+0.1625
InternLM2.5	GSM8K	84.9886	84.9886	0.0000
	LongBench v2	40.3509	40.1521	-0.1988
	MRCR	52.3585	52.2939	-0.0646

**Figure 5:** End-to-end output throughput on different datasets. The number of concurrent requests is 8.

numerical-answer accuracy on GSM8K, accuracy on LongBench v2, and token F1 on MRCR.

5.2 End-to-End Performance

5.2.1 Inference Accuracy

In Table 1, the largest score difference between SpecStream and SGLang SD is 0.1988 percentage points across the two model pairs and three tasks. SpecStream thus maintains output quality close to that of SGLang SD on the evaluated tasks. As analyzed in Section 4.2 for full causal attention, SpecStream’s streaming verification changes only the order of KV access and reduction. The different floating-point accumulation order introduced by chunked reduction may cause small numerical perturbations, resulting in differences in accuracy.

5.2.2 Performance Evaluation

Fig. 5 compares end-to-end throughput across methods. SD+Offload and SpecStream retain only the most recent 512 tokens (Active Tail) of Target KV on the GPUs. TriForce maintains a GPU retrieval cache for intermediate speculation and stores Target KV according to the offloading configuration used in this experiment. Across the three datasets, SpecStream achieves average throughput speedups of $1.41\times$ and $1.32\times$ over TriForce on Qwen3 and InternLM2.5, respectively. The corresponding speedups over SD+Offload are $1.44\times$ and

$1.62\times$. Streaming verification of SpecStream is a key source of these gains. Each incoming history KV chunk immediately serves the entire query group in the round, reducing data stalls during verification. TriForce, by contrast, reduces the frequency of full-history verification through hierarchical speculation, but incurs additional intermediate computation and retrieval-cache maintenance costs. SpecStream further improves throughput by dynamically adjusting the verification width and using spare compute capacity during transfers to advance the Draft in parallel. On the long-input LongBench v2 and MRCR workloads, however, SpecStream’s throughput remains below that of AR and SGLang SD with fully GPU-resident KV. Streaming verification can hide part of the transfer wait, but historical KV must still traverse PCIe in every verification round. The unidirectional bandwidth of PCIe Gen4 $\times$ 16 is approximately 31.5 GB/s, far below the A800’s HBM bandwidth of 1935 GB/s. Offloading the long KV history beyond the Active Tail to CPU memory reduces GPU memory usage but increases the cost of historical KV access during Target verification. The shorter inputs in GSM8K make historical KV access less costly. Here, SpecStream improves throughput over SGLang SD by 16.9% and 10.6% for the two model pairs, respectively. Its dynamic verification width increases the mean accepted length by approximately 16% and 28% for the Qwen3 and InternLM2.5 model pairs, respectively, allowing each verification round to advance more output tokens.

To examine how history offloading affects serving performance under limited GPU memory, we use a GPU-first residency policy on LongBench v2 and gradually increase the number of concurrent requests. Offloading begins only when historical KV states exceed the available capacity. After accounting for model weights and other buffers, SpecStream’s combined GPU KV pool is fixed at 40.50 GiB per GPU for Qwen3 and 52.15 GiB per GPU for InternLM2.5. TriForce applies the same GPU-first residency policy to its full Target KV Cache while retaining the retrieval cache required for intermediate speculation. Fig. 6 compares throughput across methods and reports SpecStream’s peak CPU KV history usage and GPU KV pool capacity.

Unlike AR and SGLang SD, which keep the full KV Cache on the GPUs, SpecStream and TriForce alleviate GPU memory limits by offloading historical KV states. As concurrency increases, GPU memory limits force additional requests to queue, causing AR and SGLang SD throughput to saturate. SpecStream moves the Target’s KV history to CPU memory, preserving GPU capacity for active batches. With 32 concurrent requests, its peak CPU KV history usage reaches 45.50 and 49.50 GiB per GPU for the two model pairs. This capacity benefit comes with a transfer cost. SpecStream begins offloading at 8 concurrent requests on Qwen3 and 16 on InternLM2.5. Throughput drops and then recovers in some of the subsequent concurrency configurations. At 32 concurrent requests, Qwen3 throughput is slightly higher than AR but

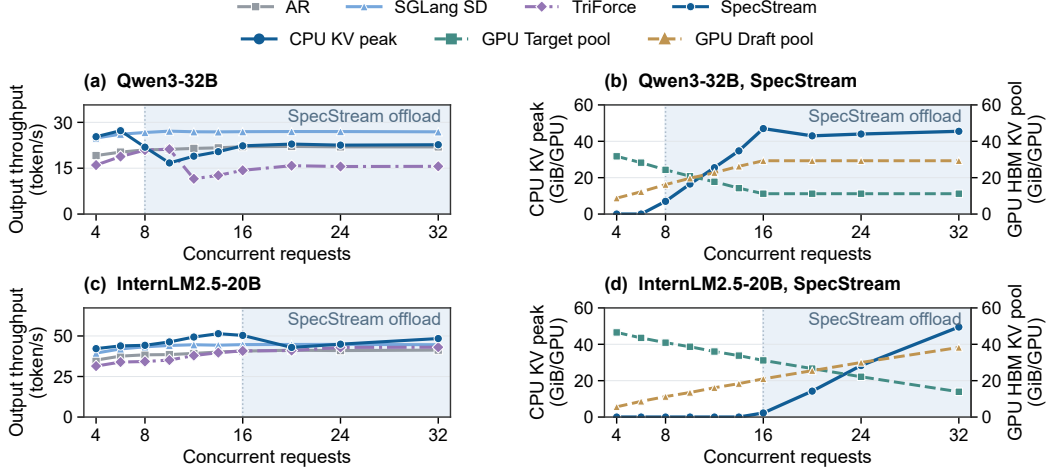


Figure 6: Performance under increasing concurrency and KV storage distribution with GPU-first residency. The left column compares end-to-end throughput. The right column shows SpecStream’s peak CPU KV history usage and GPU KV pool capacity, both reported per GPU. Shading marks the concurrency range in which SpecStream offloads KV states.

lower than SGLang SD, whereas InternLM2.5 exceeds them by 17.3% and 8.6%, respectively. Cumulative H2D traffic for verification at this point is 7.79 and 3.29 TiB per GPU for the two model pairs. Although their peak CPU KV history usage is similar, Qwen3 transfers substantially more data overall, consistent with its weaker throughput recovery.

SpecStream achieves average throughput speedups of $1.40\times$ and $1.23\times$ over TriForce on Qwen3 and InternLM2.5, respectively. TriForce’s bounded-window Draft model reduces the small model’s resident KV footprint. Its retrieval-based intermediate speculation reduces the number of full verification passes, while introducing retrieval-cache maintenance and intermediate computation. SpecStream focuses on reducing data stalls and execution overhead in full Target verification, achieving higher average throughput in these tests. It does, however, retain the complete Draft KV Cache on the GPUs. As the number of concurrent requests grows, this cache occupies an increasing share of GPU memory and becomes a non-negligible cost, even though its initial footprint is small. It therefore also limits the number of requests that SpecStream can process simultaneously on the GPUs.

5.3 Ablation Study

5.3.1 Streaming Transfers, Prefetching, and Cohort

To separate the contributions of individual mechanisms, we first disable concurrent execution on shared GPUs and progressively enable the optimizations in Table 2 within the same SpecStream implementation. K1 through K5 use identical KV budgets and inputs, with K5 representing the complete streaming verification scheme. We test 16K and 32K inputs with 1, 4, 8, and 16 concurrent requests, fixing each request’s output at 256 tokens. Fig. 7 presents the results.

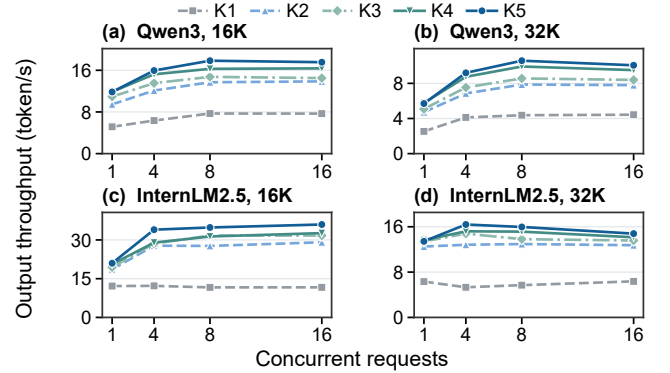


Figure 7: Throughput changes in the ablation of SpecStream’s streaming verification.

Table 2: Cumulative ablation configurations for SpecStream.

Configuration	Change from the preceding configuration	q
K1	Verify after restoring the full history	4
K2	Stream-and-reduce arriving KV chunks	4
K3	Cross-chunk and cross-layer prefetching	4
K4	Dynamic verification width	2, 4, 6, 8
K5	Merge compatible requests into a Cohort	2, 4, 6, 8

In Fig. 7, streaming access for the full query group yields the largest throughput gain. With 16K inputs, K2 improves throughput over K1 by an average of 83.2% on Qwen3 and 117.1% on InternLM2.5. Both use $q = 4$; they differ in how they organize access to the KV history. K2 uses each arriving KV chunk directly to compute attention for the entire query group, reducing fine-grained copy and submission overhead and starting verification before the full history has been restored. K3 additionally prefetches subsequent chunks and the next layer’s history. It improves throughput over K2 by

4.5% to 15.6% and 3.0% to 15.3%, respectively, showing that chunking alone leaves data stalls that prefetching can hide. K4 dynamically adjusts the verification width, changing throughput by 8.5% to 16.0% on Qwen3 and -1.2% to 9.6% on InternLM2.5. Its benefit depends on the relationship between additional computation and useful progress, which Section 5.3.2 examines further. Cohort primarily benefits from merging work across requests. With 4 to 16 concurrent requests, K5 improves throughput over K4 by 4.7% to 9.6% and 4.6% to 17.3%, respectively. It brings almost no gain with a single request, where opportunities to share batched submissions and kernel execution across compatible requests are limited. Overall, K5 achieves $2.23\times$ to $2.51\times$ and $1.73\times$ to $3.09\times$ the throughput of K1 on the two model pairs.

5.3.2 Dynamic Verification Width

Fig. 8 shows the changes in mean accepted length and throughput from K3 to K4. A longer mean accepted length does not guarantee higher throughput. On Qwen3 with 16K inputs and four requests, the mean accepted length increases by 46.8%, but throughput improves by only 12.6%. On InternLM2.5 with eight requests, the mean accepted length rises from 3.64 to 6.22, yet throughput falls by 1.2%. A larger verification width q increases computation for both drafting and verification, and rollback after rejection may introduce more wasted work. The extent to which historical KV reads can be amortized also varies with context length, batch size, and acceptance behavior. Mean accepted length alone therefore cannot determine q . SpecStream compares the estimated round time and useful progress at different values of q , seeking a width that balances the two to reduce the cost per output token.

5.3.3 Target-Priority Scheduling on Shared GPUs

We first determine the Draft model’s default TPC allocation through an offline sweep, then evaluate the overall benefit of scheduling concurrent execution on shared GPUs. For both model pairs, we measure end-to-end throughput as the Draft allocation increases from 10 to 54 TPCs per GPU. Fig. 9 shows a similar tradeoff across input lengths for both pairs. Increasing the allocation improves throughput in the low-TPC range, followed by a relatively flat region of high throughput around 30 to 38 TPCs. The highest value in this sweep occurs at 34 TPCs. This agrees with the latency trend in Fig. 1. With a small allocation, a Draft step takes longer, limiting candidate generation. Once the allocation is sufficient, further resources provide diminishing returns and may increase interference with the Target. We therefore use a default allocation of 34 TPCs per GPU in the concurrent execution experiments. At runtime, the scheduler controls when drafting starts and how much work it performs according to the Target’s state.

Fig. 10 compares three execution modes. A executes the Draft and Target sequentially, while C uses Target-priority concurrent execution on the same GPUs. Both use two GPUs with $TP=2$ for both models. B is SPECTRE+Offload, which

uses three GPUs with $TP=2$ for the Target and $TP=1$ for a dedicated Draft model. Across paired configurations with the same batch size, C improves throughput over A by 9.9% to 43.6% and 2.7% to 31.7% for the two model pairs. The relative gain is largest with a single request. With less parallel work within the batch, advancing the Draft earlier helps shorten the wait between drafting and verification. As the number of concurrent requests increases, Target batching becomes more effective, and the scheduling benefit generally narrows. This indicates that the amount of concurrent Draft work needs to adapt to the Target’s load.

Across the two model pairs, SpecStream achieves an average throughput speedup of $1.036\times$ over SPECTRE+Offload. Because SPECTRE uses one additional GPU, normalizing by GPU count gives SpecStream an average improvement of 55.4% in output throughput per GPU. Both schemes overlap drafting with Target verification, but they use different resources. SpecStream reuses spare compute capacity while the Target waits for historical KV. Execution on separate GPUs provides dedicated compute resources for the Draft model, while requiring candidates and verification results to be exchanged across devices. Once candidates can be supplied on time, faster drafting need not shorten a round dominated by historical KV reads and Target verification. Scheduling concurrent execution on shared GPUs can therefore sustain comparable throughput with fewer devices and improve average output throughput per GPU in this experimental configuration.

5.4 Discussion

The results show that SpecStream alleviates capacity limits by tiering historical KV states and improves throughput under offloading through multi-query streaming verification and scheduling on shared GPUs. Streaming verification promptly uses each arriving history chunk for all queries in the round, reducing exposed data stalls. Concurrent scheduling advances the Draft with spare compute capacity during transfers, allowing two GPUs to achieve throughput close to three-GPU SPECTRE+Offload. These gains come from coordinating KV access with model execution. However, Fig. 6 also shows that expanding the KV pool for the Draft reduces the GPU residency budget for the Target’s KV. The current implementation retains the complete Draft KV on GPU, because consecutive Draft steps lack the opportunity for multi-query reuse within a Target verification round. Directly applying the same offloading path could require repeatedly fetching historical KV across consecutive steps, increasing the wait for candidate generation. Expanding the active batch further therefore calls for Draft KV management that uses less GPU memory.

6 Related Work

Speculative decoding and verification optimization. Speculative decoding reduces sequential Target invocations through inexpensive drafting and batched verification [4, 5]. Medusa

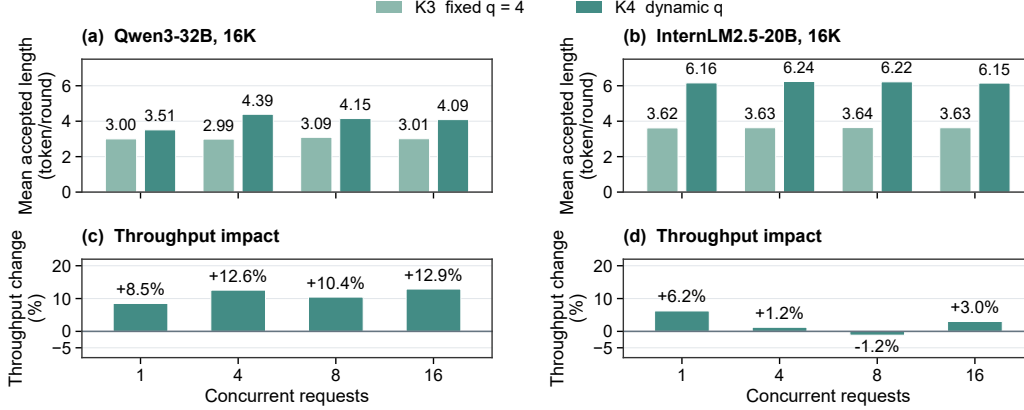


Figure 8: Effect of dynamic verification width on mean accepted length and throughput. The top row compares mean accepted lengths for K3 and K4; the bottom row reports K4’s throughput change relative to K3.

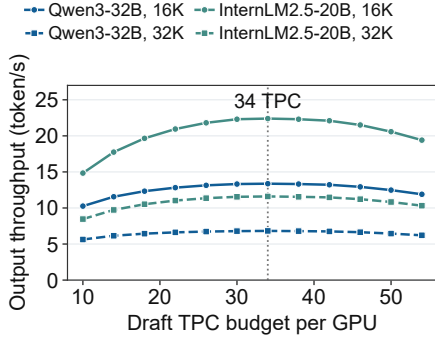


Figure 9: Effect of the Draft TPC allocation on throughput.

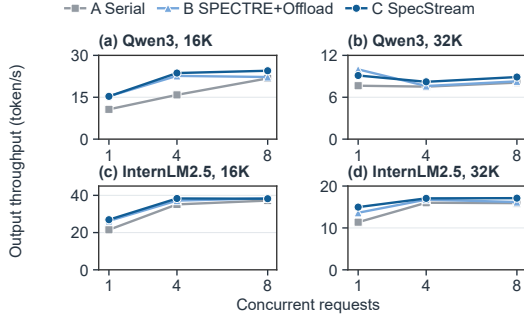


Figure 10: End-to-end output throughput of sequential execution, SPECTRE+Offload, and SpecStream.

[7], the EAGLE family [6, 26, 27], SpecInfer [8], and Sequoia [28] improve useful output per round through candidate prediction and tree-based verification. MagicDec [29] and QuantSpec [30] reduce long-context drafting overhead using sparse KV states or quantization. DeFT [31] groups queries and KV states around shared prefixes to reduce redundant KV reads within the GPU.

KV management and offloading. Paged allocation [2], prefix sharing [21], and context caching [9, 32, 33] reduce wasted KV storage and repeated prefill computation. Flex-

Gen [3] jointly plans the tiered placement of weights, activations, and KV states. HeadInfer [34] offloads KV states at attention-head granularity, whereas InfiniGen [10] and SpecCache [35] selectively fetch important KV states to reduce data movement. Recent work also combines offloading with speculative decoding. TriForce [25] builds hierarchical speculation around retrieval-based drafting and amortizes the cost of fetching the full history through multi-token verification. VeriCache [36] drafts with compressed KV states and fetches the full Target KV Cache for verification. SpecOffload [37] targets throughput-oriented inference when model weights cannot fit entirely in GPU memory and places Target weights and KV states across memory tiers. Dovetail [38] assigns Target verification to the CPU and drafting to the GPU, coordinating heterogeneous execution under limited GPU resources.

Parallel speculation and serving schedules. General-purpose serving systems improve resource utilization through iteration-level batching [39], chunked prefill [40], and phase disaggregation [41]. MineDraft [11] overlaps drafting and verification through a pipeline across batches, SwiftSpec [13] optimizes disaggregated execution and data exchange, PEARL [42] overlaps Draft and Target execution via pre-verify and post-verify, and SPECTRE [12] reuses remote Draft services.

In contrast, SpecStream combines within-round multi-query sharing, chunked KV transfers, and online softmax, allowing full-attention Target verification to proceed before restoring the current layer’s full KV history. It also exploits compute bubbles caused by Target KV-transfer waits to advance the Draft on the same GPUs, enabling parallel speculation without an additional Draft GPU.

Conclusion

We present SpecStream, which jointly designs history management, streaming computation, and scheduling on shared GPUs around the state and access characteristics of the KV

Cache in speculative decoding. The commit boundary separates history migration from candidate rollback. Queries within a round share the full history as it arrives in chunks, while Target-priority scheduling advances the Draft during data waits. Our experiments show that organizing KV access and model execution around the complete speculative decoding step can both alleviate GPU memory limits and improve serving efficiency under offloading. In future work, we plan to extend KV offloading beyond CPU memory to additional storage tiers.

References

- [1] Tom B. Brown, Benjamin Mann, Nick Ryder et al. Language Models are Few-Shot Learners. *Advances in Neural Information Processing Systems (NeurIPS 2020)*, vol. 33, pp. 1877–1901.
- [2] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP 2023)*, pp. 611–626.
- [3] Ying Sheng, Lianmin Zheng, Binhang Yuan et al. FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU. *Proceedings of the 40th International Conference on Machine Learning (ICML 2023)*, vol. 202, pp. 31094–31116.
- [4] Yaniv Leviathan, Matan Kalman, Yossi Matias. Fast Inference from Transformers via Speculative Decoding. *Proceedings of the 40th International Conference on Machine Learning (ICML), 2023*, vol. 202, pp. 19274–19286.
- [5] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving et al. Accelerating Large Language Model Decoding with Speculative Sampling. *arXiv preprint arXiv:2302.01318*, 2023.
- [6] Yuhui Li, Fangyun Wei, Chao Zhang et al. EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty. *Proceedings of the 41st International Conference on Machine Learning (ICML), 2024*, vol. 235, pp. 28935–28948.
- [7] Tianle Cai, Yuhong Li, Zhengyang Geng et al. Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads. *Proceedings of the 41st International Conference on Machine Learning (ICML), 2024*, vol. 235, pp. 5209–5235.
- [8] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang et al. SpecInfer: Accelerating Large Language Model Serving with Tree-based Speculative Inference and Verification. *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2024*, vol. 3, pp. 932–949.
- [9] Bin Gao, Zhuomin He, Puru Sharma et al. Cost-Efficient Large Language Model Serving for Multi-turn Conversations with CachedAttention. *2024 USENIX Annual Technical Conference (USENIX ATC 2024)*, pp. 111–126.
- [10] Wonbeom Lee, Jungi Lee, Junghwan Seo et al. InfiniGen: Efficient Generative Inference of Large Language Models with Dynamic KV Cache Management. *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2024)*, pp. 155–172.
- [11] Zhenwei Tang, Arun Verma, Zijian Zhou et al. MineDraft: A Framework for Batch Parallel Speculative Decoding. *International Conference on Machine Learning (ICML), 2026*. *arXiv:2603.18016*.
- [12] Jincheng Xie, Yawen Ling, Qi Xiao et al. SPECTRE: Hybrid Ordinary-Parallel Speculative Serving for Resource-Efficient LLM Inference. *arXiv preprint arXiv:2605.08151*, 2026.
- [13] Ziyi Zhang, Ziheng Jiang, Chengquan Jiang et al. SwiftSpec: Disaggregated Speculative Decoding and Fused Kernels for Low-Latency LLM Inference. *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2026*, vol. 2, pp. 2197–2211.
- [14] An Yang, Anfeng Li, Baosong Yang et al. Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388*, 2025.
- [15] InternLM Team. InternLM2.5-20B-Chat. Official Hugging Face model card. Accessed 2026-08-25.
- [16] Joshua Bakita, James H. Anderson. Hardware Compute Partitioning on NVIDIA GPUs. *Proceedings of the 29th IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS), 2023*, pp. 54–66.
- [17] Tri Dao, Daniel Y. Fu, Stefano Ermon et al. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *Advances in Neural Information Processing Systems (NeurIPS 2022)*, vol. 35, pp. 16344–16359.
- [18] Tri Dao. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. *The Twelfth International Conference on Learning Representations (ICLR 2024)*.
- [19] NVIDIA. Multi-Process Service. *NVIDIA CUDA Documentation*. Accessed 2026-09-12.
- [20] NVIDIA. NVIDIA A100 Tensor Core GPU Architecture. *Architecture Whitepaper, 2020*. Accessed 2026-09-12.
- [21] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie et al. SGLang: Efficient Execution of Structured Language Model Programs. *Advances in Neural Information Processing Systems (NeurIPS 2024)*, vol. 37, pp. 62557–62583.
- [22] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian et al. Training Verifiers to Solve Math Word Problems. *arXiv preprint arXiv:2110.14168*, 2021.
- [23] Yushi Bai, Shangqing Tu, Jiajie Zhang et al. LongBench v2: Towards Deeper Understanding and Reasoning on

- Realistic Long-context Multitasks. Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 2025, vol. 1, pp. 3639–3664.
- [24] OpenAI. OpenAI MRCR: Long context multiple needle in a haystack benchmark. Hugging Face dataset card, 2025.
 - [25] Hanshi Sun, Zhuoming Chen, Xinyu Yang et al. Tri-Force: Lossless Acceleration of Long Sequence Generation with Hierarchical Speculative Decoding. First Conference on Language Modeling (COLM), 2024.
 - [26] Yuhui Li, Fangyun Wei, Chao Zhang et al. EAGLE-2: Faster Inference of Language Models with Dynamic Draft Trees. Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 7421–7432.
 - [27] Yuhui Li, Fangyun Wei, Chao Zhang, Hongyang Zhang. EAGLE-3: Scaling up Inference Acceleration of Large Language Models via Training-Time Test. Advances in Neural Information Processing Systems (NeurIPS), 2025, vol. 38, pp. 136737–136756.
 - [28] Zhuoming Chen, Avner May, Ruslan Svirschevski et al. Sequoia: Scalable and Robust Speculative Decoding. Advances in Neural Information Processing Systems (NeurIPS), 2024, vol. 37, pp. 129531–129563.
 - [29] Ranajoy Sadhukhan, Jian Chen, Zhuoming Chen et al. MagicDec: Breaking the Latency-Throughput Tradeoff for Long Context Generation with Speculative Decoding. The Thirteenth International Conference on Learning Representations (ICLR), 2025.
 - [30] Rishabh Tiwari, Haocheng Xi, Aditya Tomar et al. QuantSpec: Self-Speculative Decoding with Hierarchical Quantized KV Cache. Proceedings of the 42nd International Conference on Machine Learning (ICML), 2025, vol. 267, pp. 59668–59686.
 - [31] Jinwei Yao, Kaiqi Chen, Kexun Zhang et al. DeFT: Decoding with Flash Tree-attention for Efficient Tree-structured LLM Inference. The Thirteenth International Conference on Learning Representations (ICLR), 2025.
 - [32] Jiayi Yao, Hanchen Li, Yuhua Liu et al. CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion. Proceedings of the Twentieth European Conference on Computer Systems (EuroSys 2025), pp. 94–109.
 - [33] Yang Liu, Yunfei Gu, Liqiang Zhang et al. CacheSlide: Unlocking Cross Position-Aware KV Cache Reuse for Accelerating LLM Serving. 24th USENIX Conference on File and Storage Technologies (FAST 2026), pp. 83–99.
 - [34] Cheng Luo, Zefan Cai, Hanshi Sun et al. HeadInfer: Memory-Efficient LLM Inference by Head-wise Offloading. arXiv preprint arXiv:2502.12574, 2025.
 - [35] Shibo Jie, Yehui Tang, Kai Han et al. SpeCache: Speculative Key-Value Caching for Efficient Generation of LLMs. Proceedings of the 42nd International Conference on Machine Learning (ICML), 2025, vol. 267, pp. 27917–27928.
 - [36] Jiayi Yao, Samuel Shen, Kuntai Du et al. VeriCache: Turning Lossy KV Cache into Lossless LLM Inference. arXiv preprint arXiv:2605.17613, 2026.
 - [37] Xiangwen Zhuge, Shen Xu, Zeyu Wang et al. SpecOfload: Unlocking Latent GPU Capacity for LLM Inference on Resource-Constrained Devices. arXiv preprint arXiv:2505.10259, 2025.
 - [38] Libo Zhang, Zhaoning Zhang, Baizhou Xu et al. Dove-tail: A CPU/GPU Heterogeneous Speculative Decoding for LLM inference. Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 17382–17395.
 - [39] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim et al. Orca: A Distributed Serving System for Transformer-Based Generative Models. 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2022), pp. 521–538.
 - [40] Amey Agrawal, Nitin Kedia, Ashish Panwar et al. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2024), pp. 117–134.
 - [41] Yinmin Zhong, Shengyu Liu, Junda Chen et al. Dist-Serve: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2024), pp. 193–210.
 - [42] Tianyu Liu, Yun Li, Qitan Lv, Kai Liu, Jianchen Zhu, Winston Hu, and Xiao Sun. PEARL: Parallel Speculative Decoding with Adaptive Draft Length. The Thirteenth International Conference on Learning Representations (ICLR 2025).